

UE-INF2468M Modélisation Mondes Virtuels



Université Claude Bernard



Lyon 1

COTTIER Alexandre, P2018147 : <https://github.com/Mantador01/mmv>

J'ai implémenté une liste de choses assez intéressante au visuel (pour ce tp je me suis vraiment intéressé à expérimenter plusieurs choses plutôt que d'optimiser et pousser au plus loin une ou deux features)

1) Snow cover (simulation climatique) :

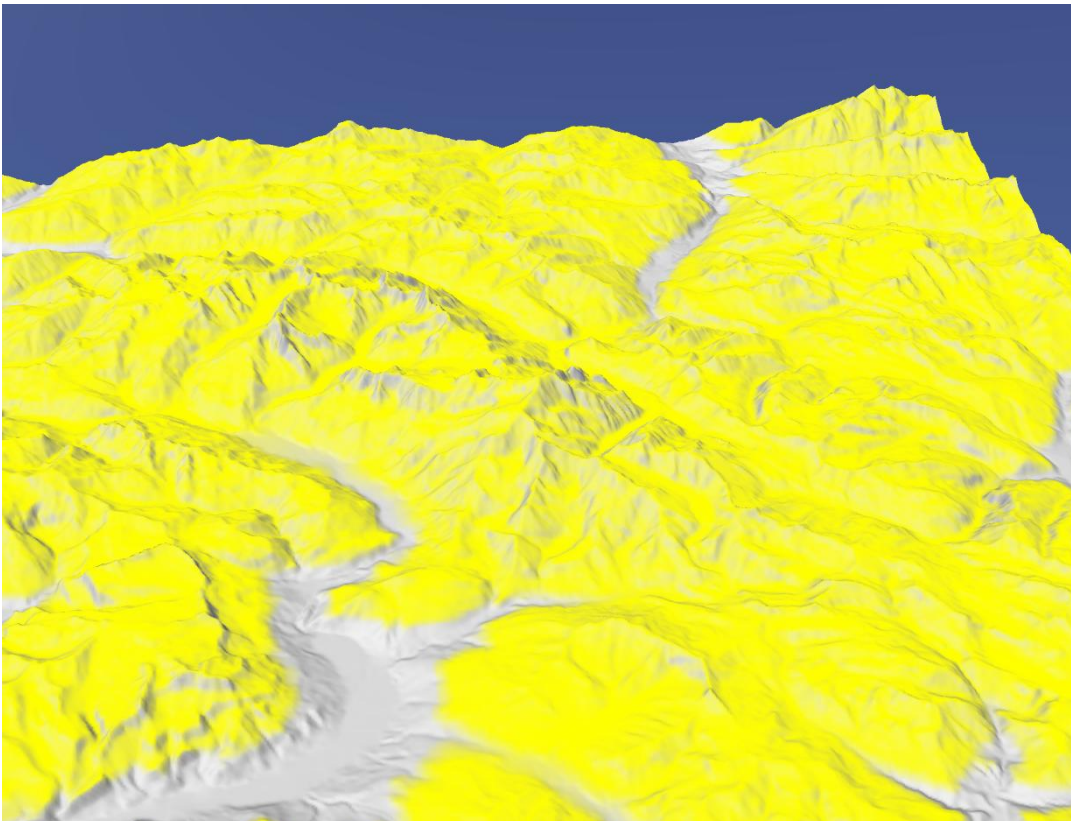
Le principe de base : La neige apparaît si Altitude > Seuil ET si Pente < Limite.

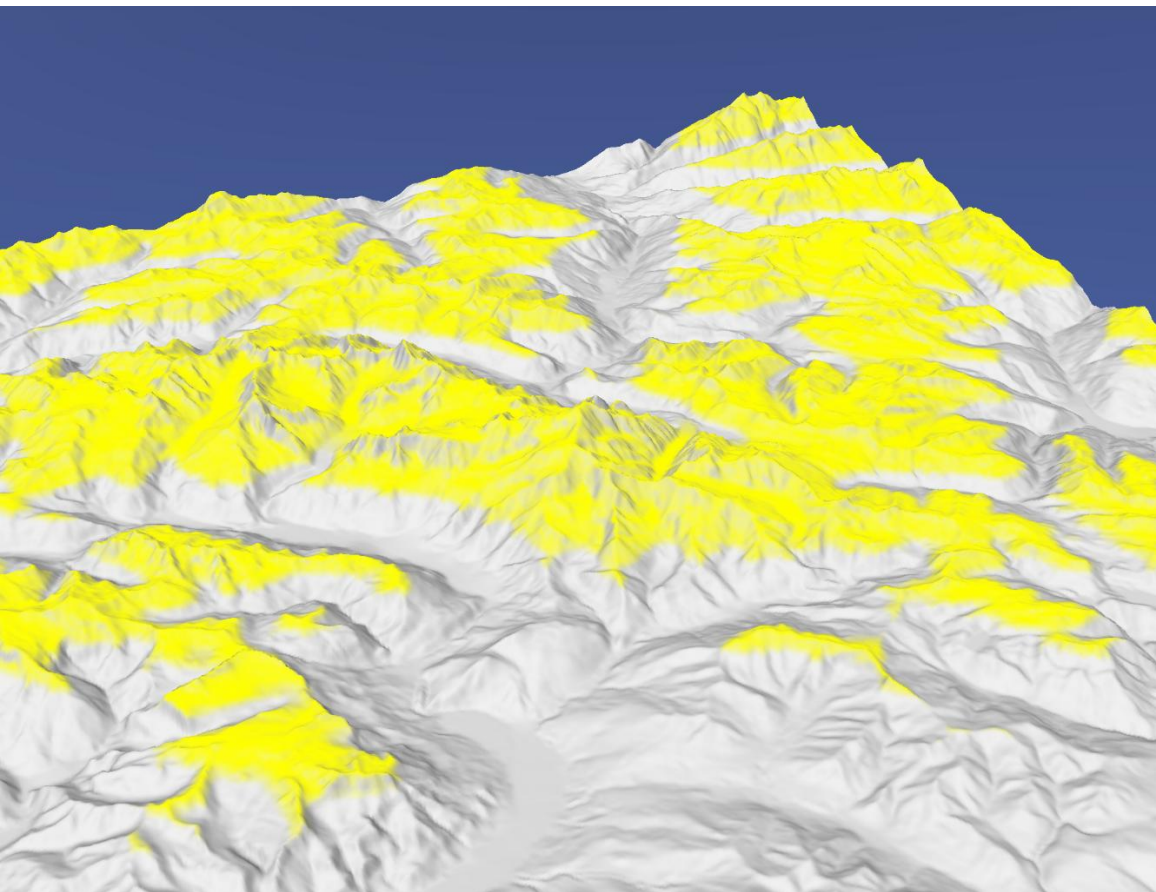
On obtient des résultats comme :

(J'ai coloré en jaune parce qu'en blanc on ne voit pas trop)

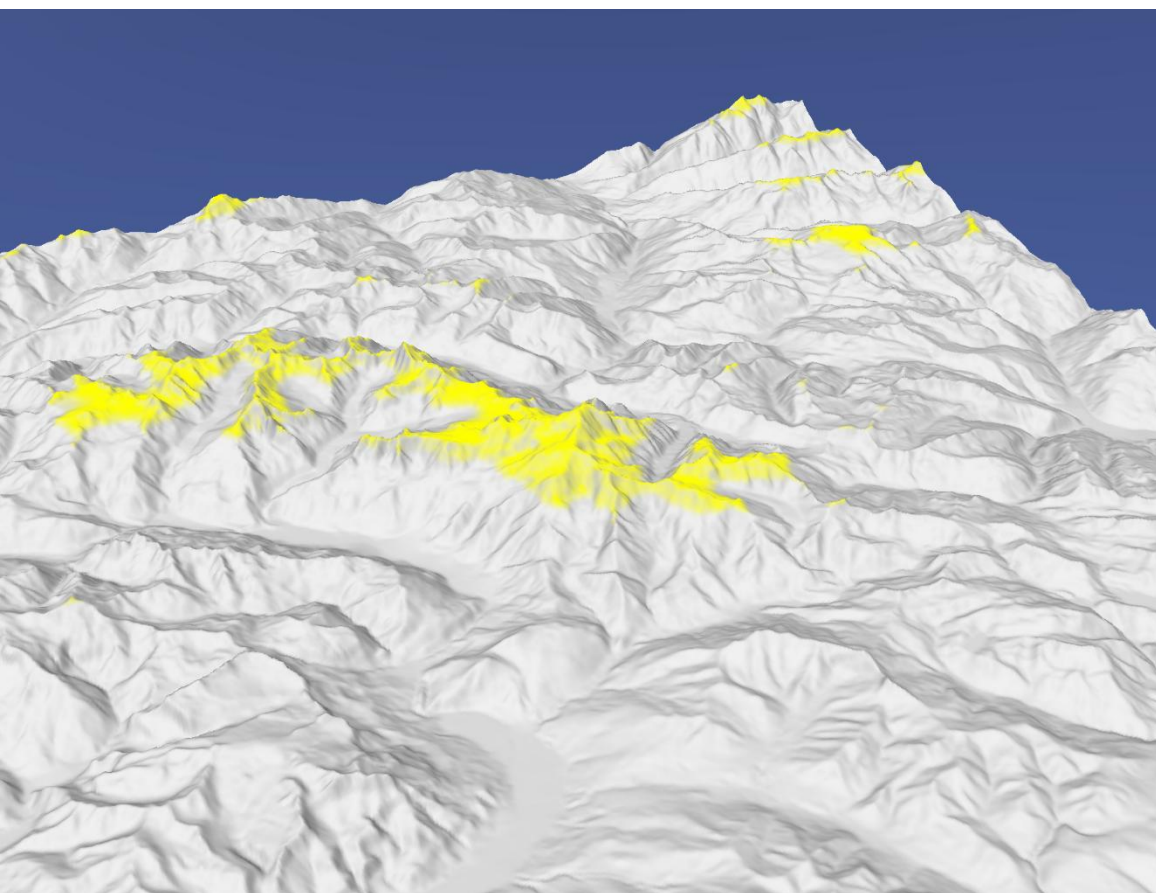
SnowLine en mètres

1000m





2000m



3000m

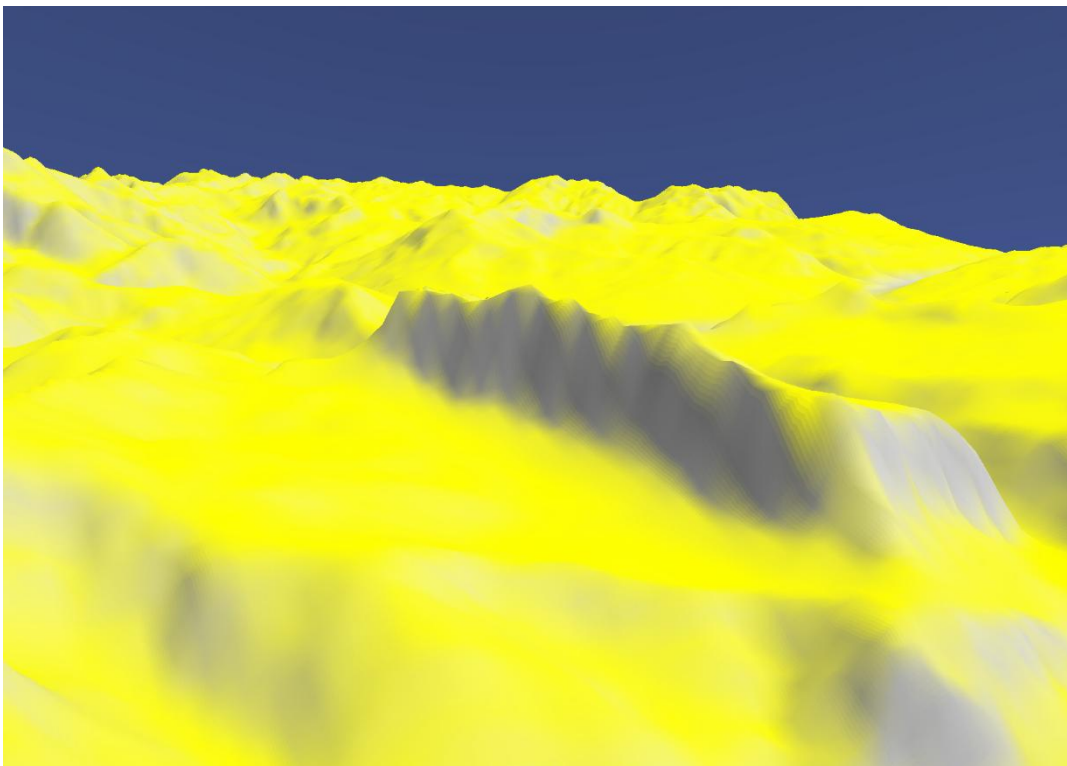
Ici on a un comportement semi-physique où la neige dépend de deux facteurs environnementaux, la température (liée à l'altitude) et la gravité (liée à la pente).

L'algorithme (sans rentrer dans les gros détails) :

Pour chaque point je calcul un coefficient de présence de neige [0,1] basé sur les deux critères cités au-dessus :

- **L'altitude** : je définis une limite (SnowLine), en dessous de cette limite la neige est absente. Au-dessus, la neige apparait progressivement (je fais une zone de transition du style fade-in) sur 200m pour éviter que l'on voie littéralement une coupure nette.
- **La Pente** : Pour la simulation on ne veut pas voir de la neige s'accumuler sur les parois verticales des montagnes par exemple, du coup j'utilise la normale du terrain pour déduire la pente locale, si la surface est trop pentue, la neige glisse et laisse apparaître du coup le terrain de base sans neige (c'est quand on voit les bouts blancs sur les pics)

Le coefficient final c'est juste le produit de ces deux facteurs.



Ici on peut voir les parties blanches qui correspond à quand on a une pente beaucoup trop raide.

Je pense que c'est assez limpide cette partie donc je ne pense pas que ce soit nécessaire de montrer des captures d'écran avec plusieurs valeurs pour la pente accepter, on se rend vite compte que si on accepte plus alors c'est juste plus enneiger et plus moche car moins réaliste (et ressemble juste à un simple color de la map)

2) Infrastructure Routière et Terrassement :

Le principe de base : Je voulais générer un réseau routier qui relie deux points donnés.

1. Je trouve le chemin optimal en évitant les pentes trop raides (Pathfinding).
2. J'adapte le terrain à la route en creusant (déblai) ou en surélevant (remblai) la chaussée pour qu'elle soit plate.

L'algorithme (sans rentrer dans les gros détails) :

Pour trouver le chemin, j'utilise A* sur la grille du terrain. Pour ma fonction de cout je ne cherche pas que la distance la plus courte, mais le chemin qui minimise l'effort de montée.

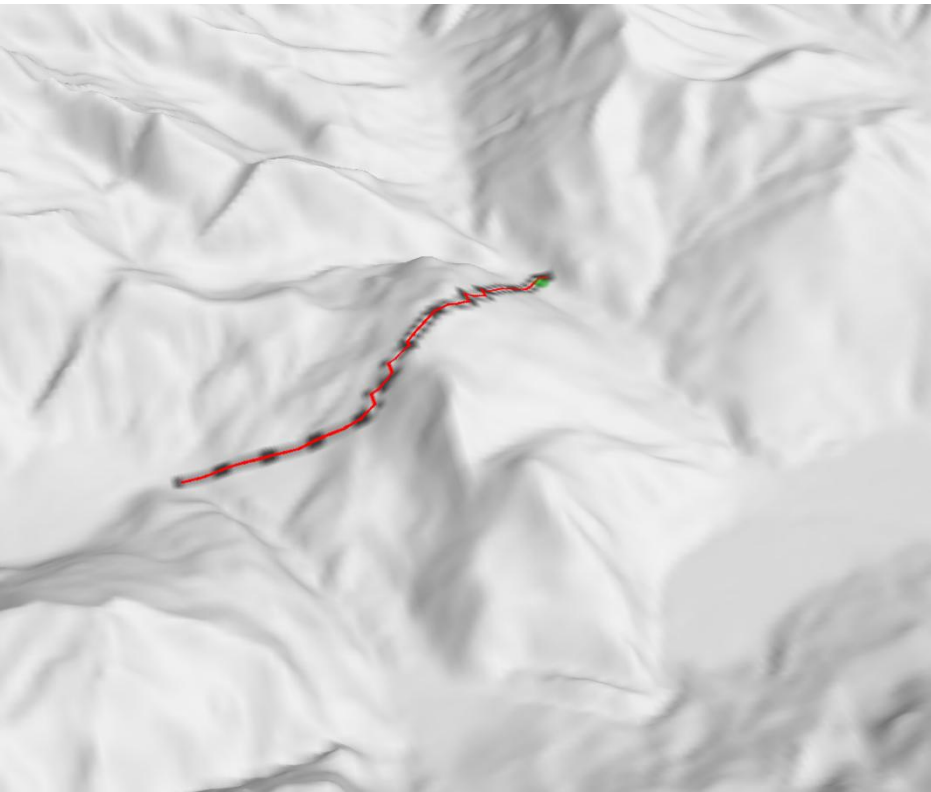
$$\text{Cost} = \text{Distance} + (k * \text{Pente})$$

Où k est un facteur de pénalité. Si k est élevé, alors on va faire un long détour plutôt que de monter une pente raide face à la pente.

Une fois le chemin tracé, pour chaque point du terrain proche de la route, nous calculons sa nouvelle hauteur par interpolation.

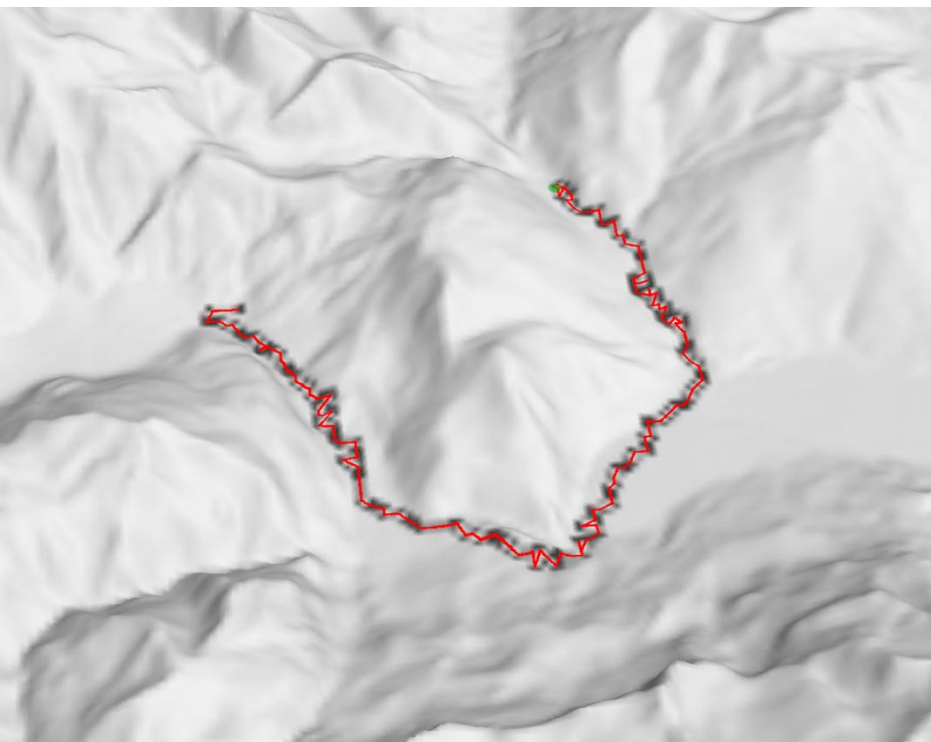
- Zone Chaussée : La hauteur est forcée à celle de la route (aplatissement).
- Zone Talus (Blending) : Je fais une transition douce (fonction Cosinus) entre la route et le terrain.

Exemples et comparaisons :



Tracé minimisant uniquement la distance (Pénalité pente = 0). La route est impraticable (entre guillemets).

Ici mon vois bien que la route grimpe tout droit sur le pic, si on veut être réaliste on peut dire que pour une voiture ce sera surement trop raide par exemple.



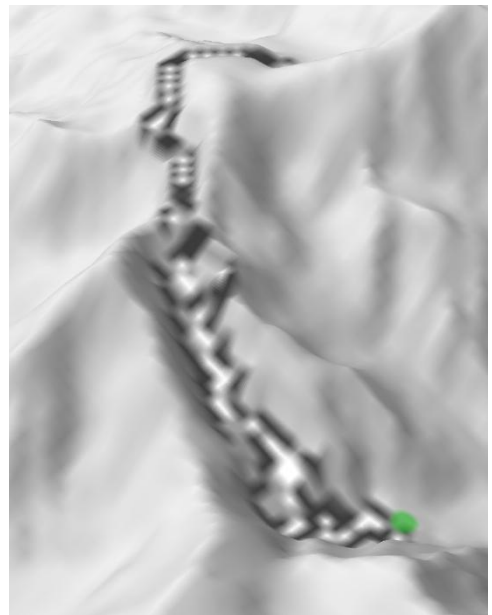
Tracé prenant en compte la topographie (Pénalité pente élevée). On contourne les gros obstacles.

La route va contourner la montagne / cherche un col plus bas. Elle fait des détours logiques et réaliste.

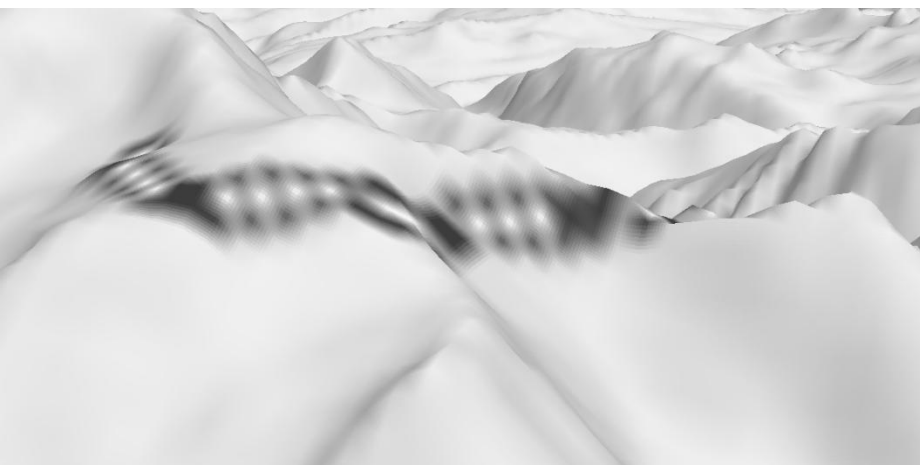
Pour aller plus loin on voit que les routes sont assez moches (beaucoup de petit zig zag) j'avais commencé rapidement à faire un smoothPath. (pas fini)



Vue d'en haut

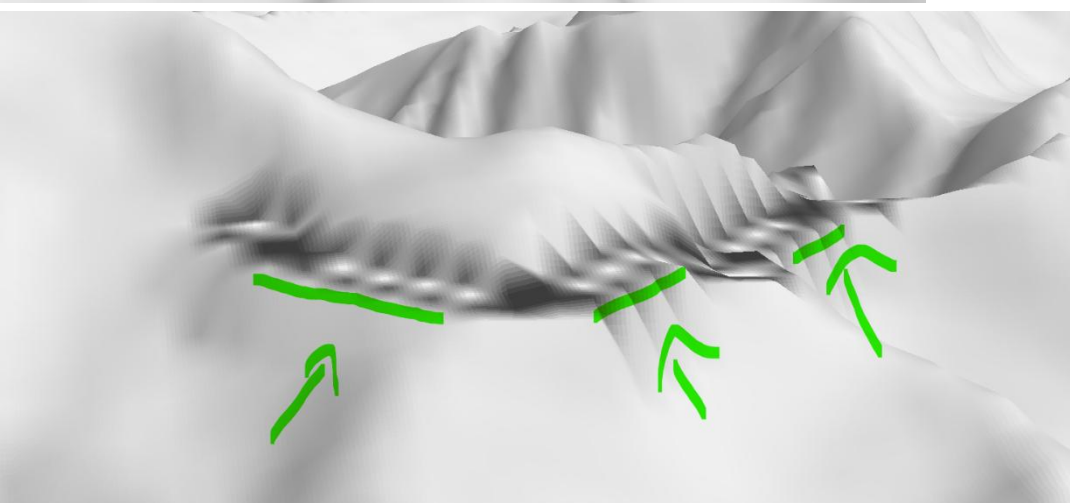


Ici je décide de terraformer ma route, on peut voir sur la deuxième image que je garde de dessiner les contours de la route, et que cela à surélever le terrain aux endroit nécessaire et creuser dans le terrain quand cela été nécessaire aussi (de haut cela ne se voit pas trop)



Vue d'en bas

Avant terraforming



Vue d'en bas

**Après
terraforming**

3) Génération procédurale de civilisation et la simulation de trafic :

Le principe de base : Je voulais automatiser la création de ville (et les routes les reliant), l'algorithme identifie les zones habitables et va y placer des villes (5 villes ici mais je peux en mettre autant que je veux/possible), les relier intelligemment et simuler un trafic sur le réseau (des voitures).

Placement des villes :

Pour places les villes j'utilise l'approche de *Monte Carlo* (Sampling) avec contraintes, je tire des positions aléatoires et je les valide que si elles respectent des critères géographiques stricts comme :

- 1) L'altitude : La ville ne doit pas être trop bas (sous l'eau par ex).
- 2) La Pente : Le terrain doit être plutôt plat (pente < 10%).
- 3) La Densité : Une distance minimale (ex 1km) doit être respectée entre deux villes pour éviter des chevauchements.

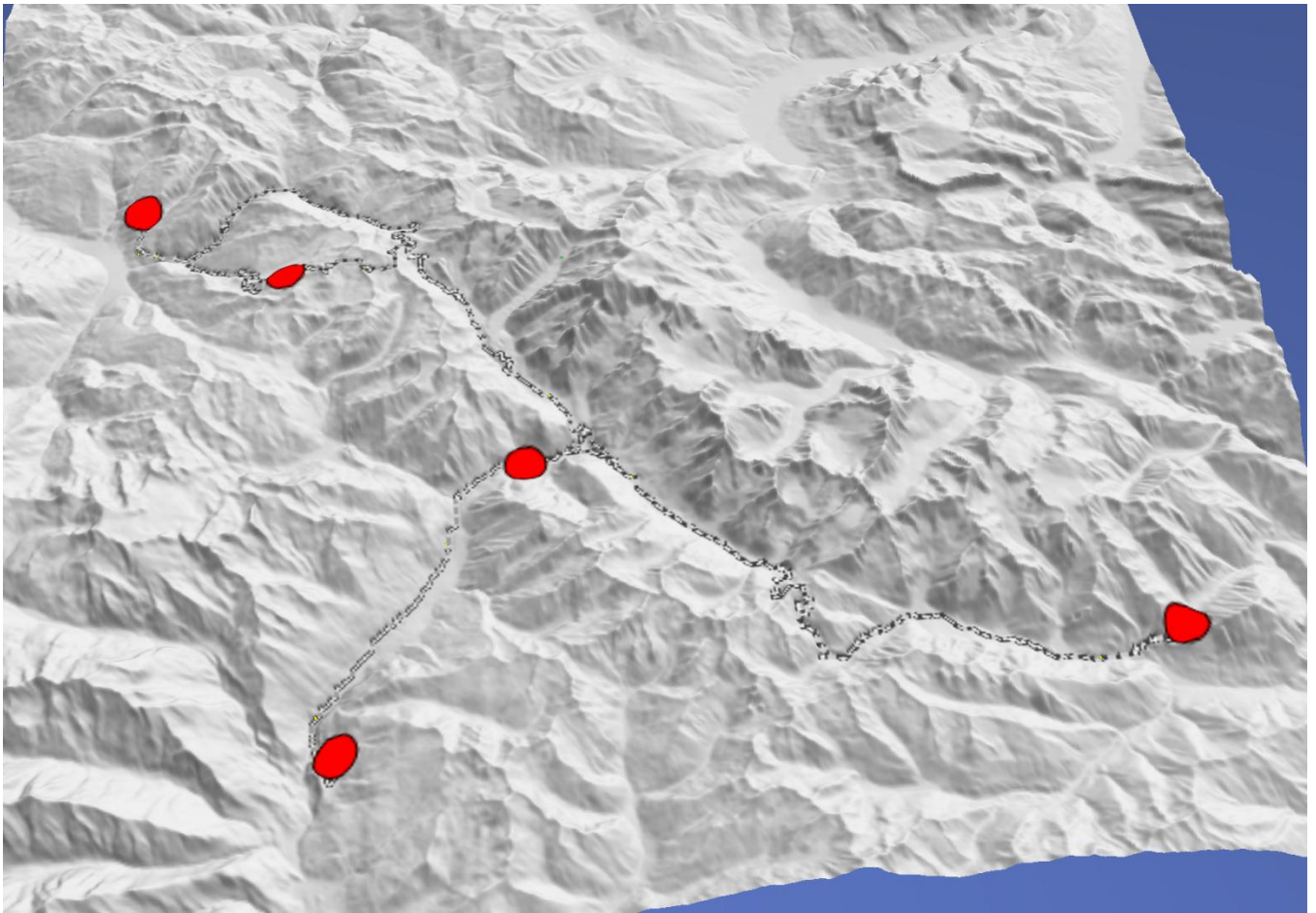
Construction du Réseau Routier :

- Topologie : Je fais un cycle simple reliant chaque ville à la suivante.
- Tracé : Pour chaque segment, j'appelle le module de Pathfinding A*.
- Terraforming Automatique : Une fois le tracé calculé, j'applique automatiquement les opérations de Déblai/Remblai pour creuser la montagne et créer une plateforme routière.

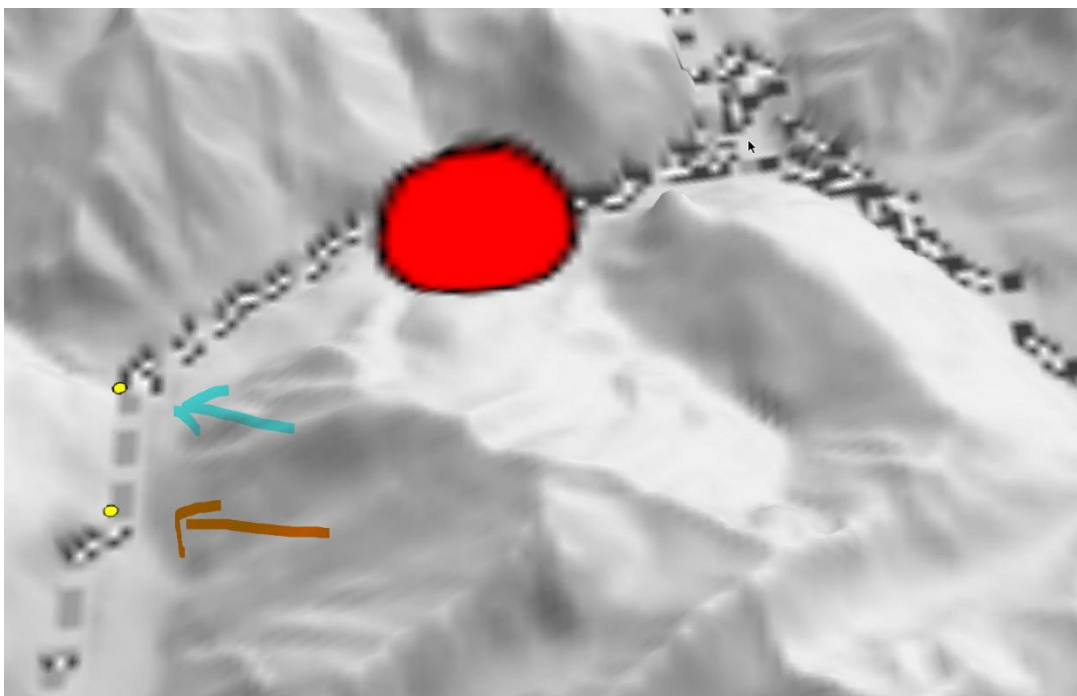
Simulation d'Agents (Trafic) :

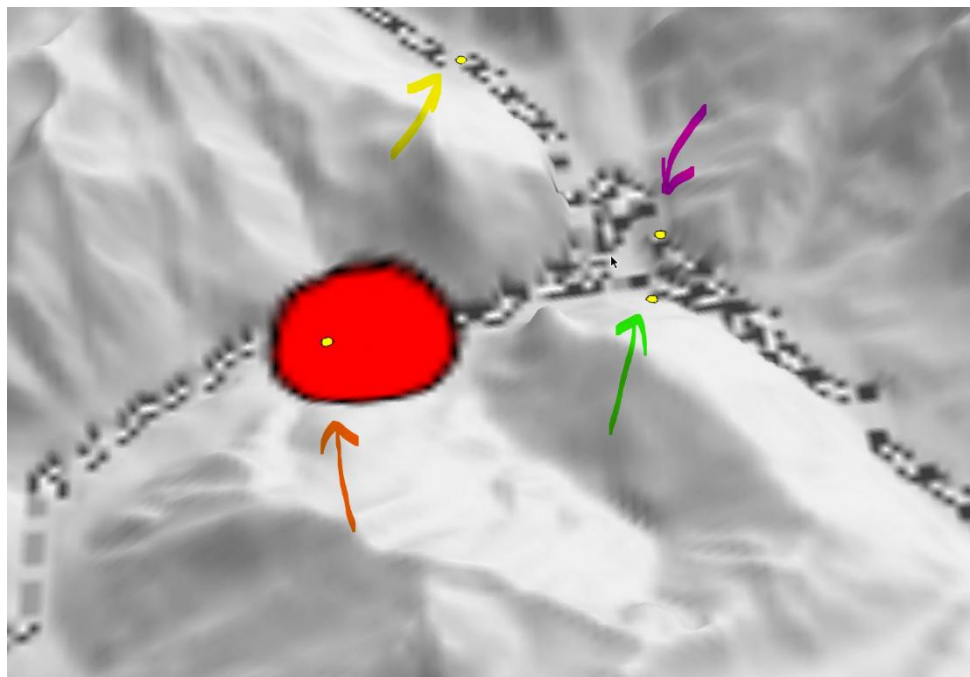
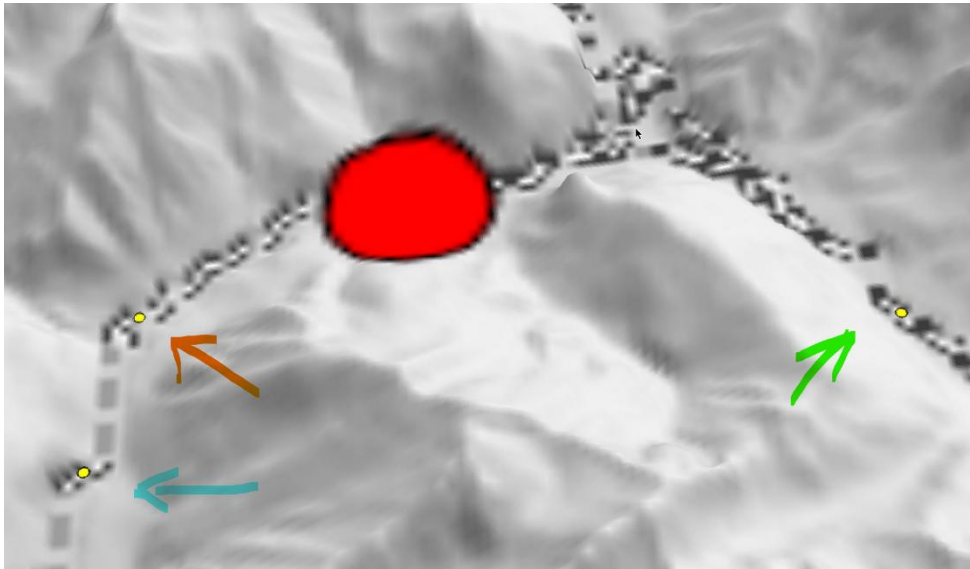
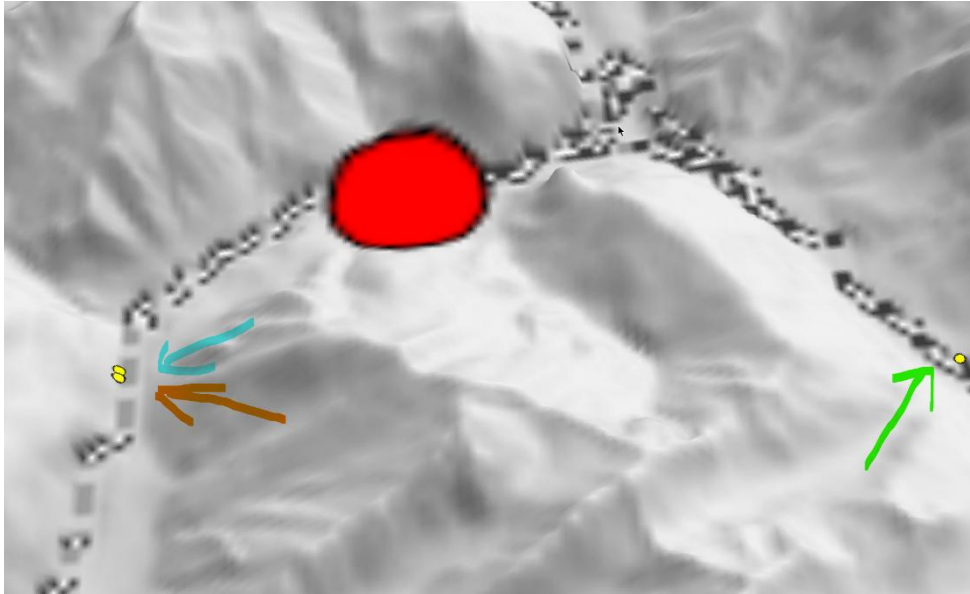
- Chaque voiture est une entité qui possède une position et une destination.
- Elles apparaissent dans les villes et suivent les chemins calculés précédemment.
- Un cycle de mise à jour déplace les agents le long de la courbe.

Exemples et comparaisons :



Réseau routier généré procéduralement. Les villes (en rouge) sont situées dans les zones plates. Les routes (en gris) relient les villes en s'adaptant à la topographie.





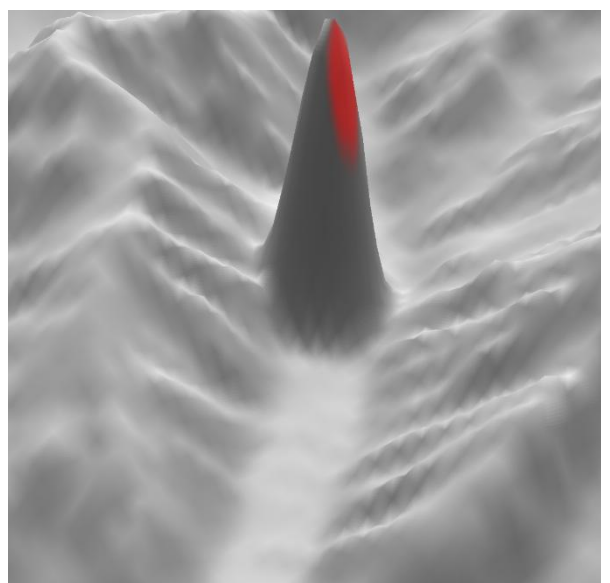
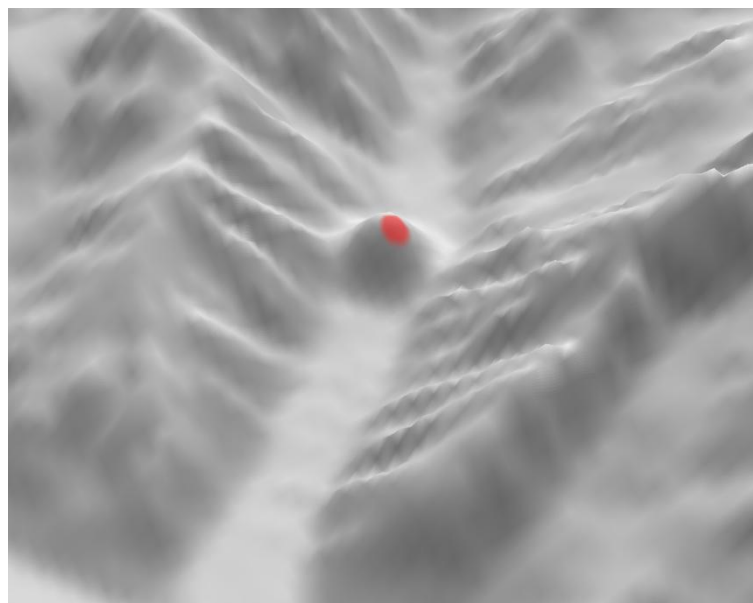
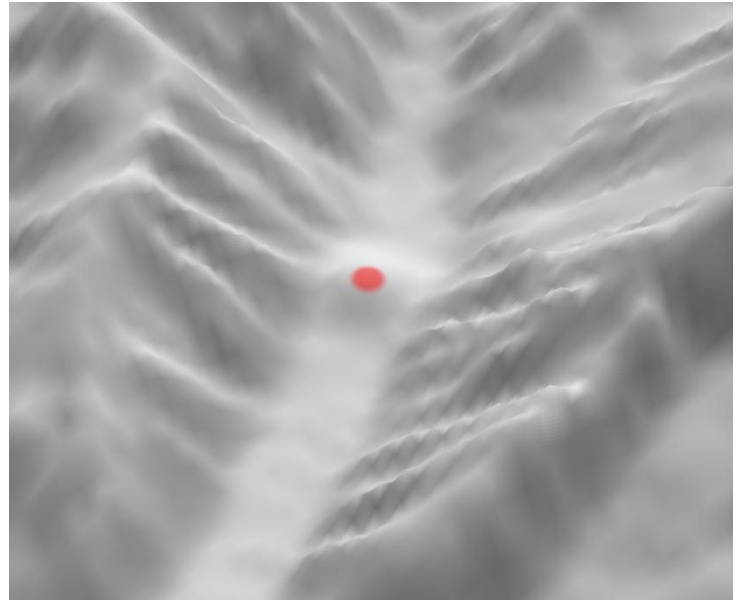
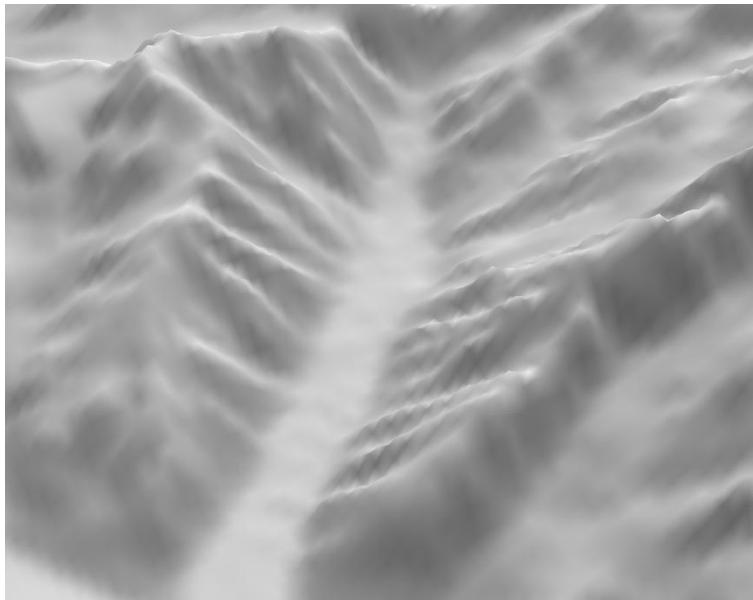
*Simulation de trafic
en temps réel. Les
agents (points
jaunes) naviguent de
manière autonome
entre les villes sur le
réseau construit.*

4) God Mode (on sculpte le terrain a la main)

Le principe de base : C'est pendant que je faisais le tp que je me suis dit que ça pourrait être intéressant pour la suite de pouvoir carve un peu le terrain a la main, j'ai donc fait un outil d'édition pour altérer la topographie avec la souris (comme pinceau).

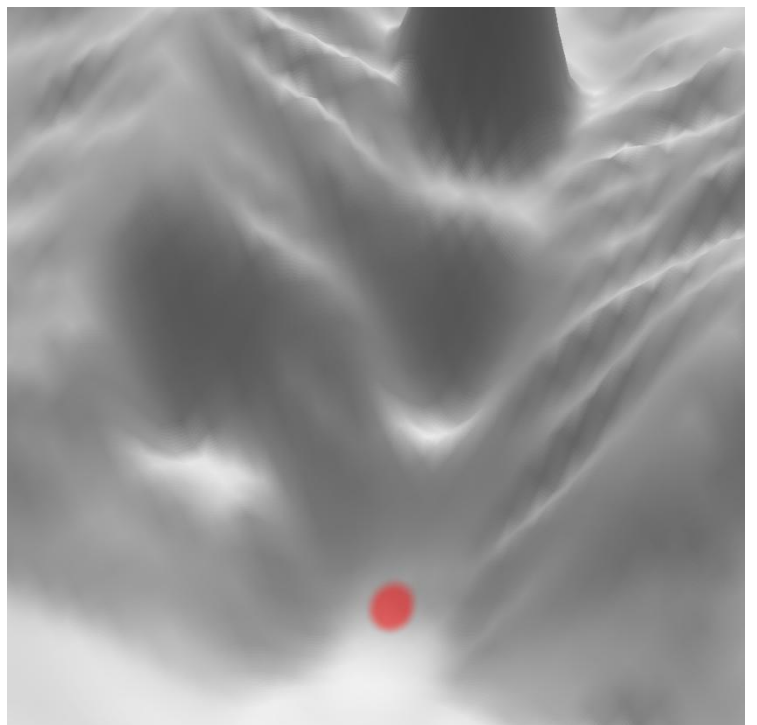
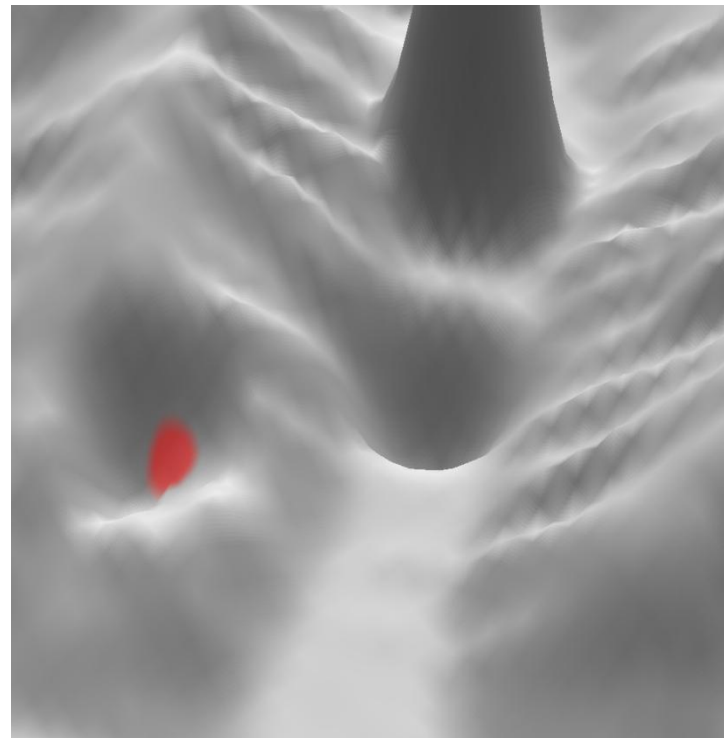
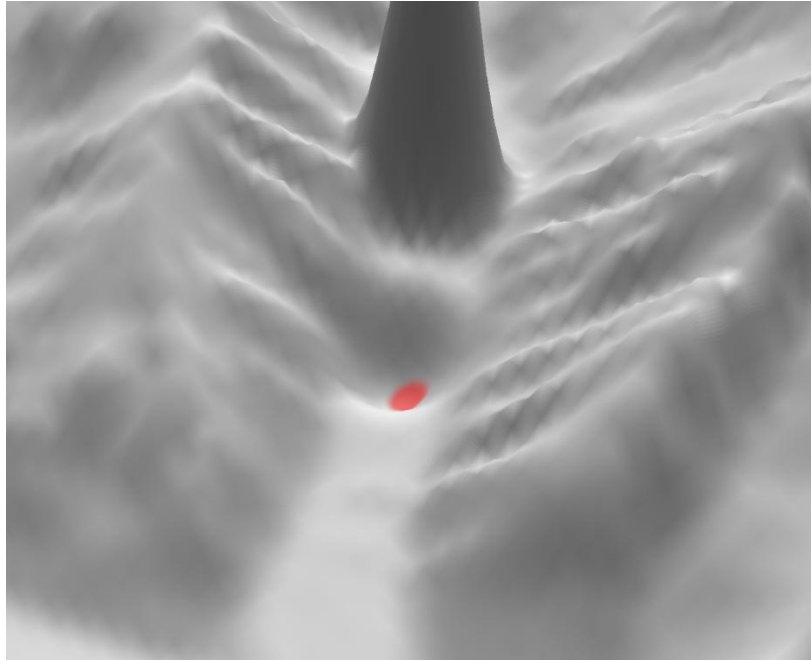
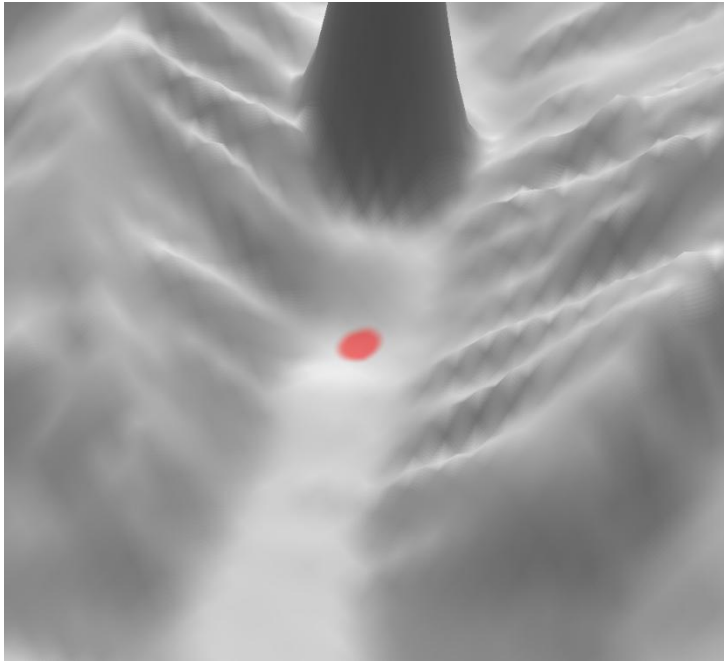
Il n'y a rien de très lié a ce cours ci mais je fais simplement à chaque clic droit je projette un rayon dans l'espace 3D, je calcule l'intersection du rayon avec le plan du terrain pour avoir le point d'impact.

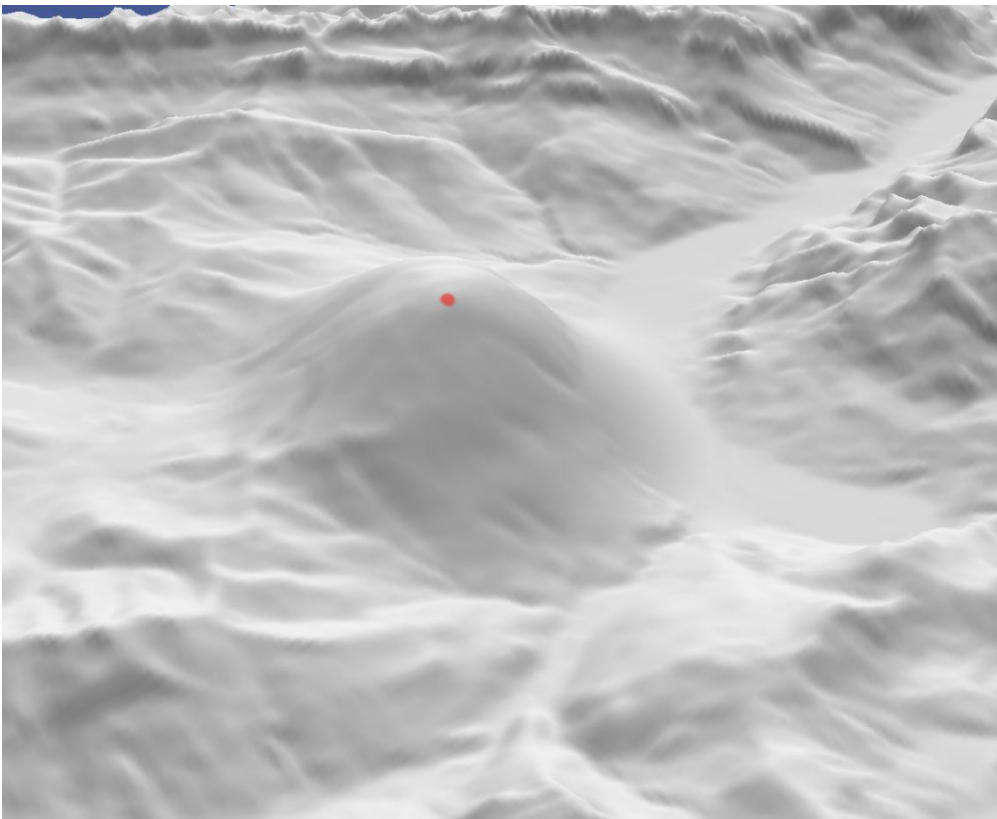
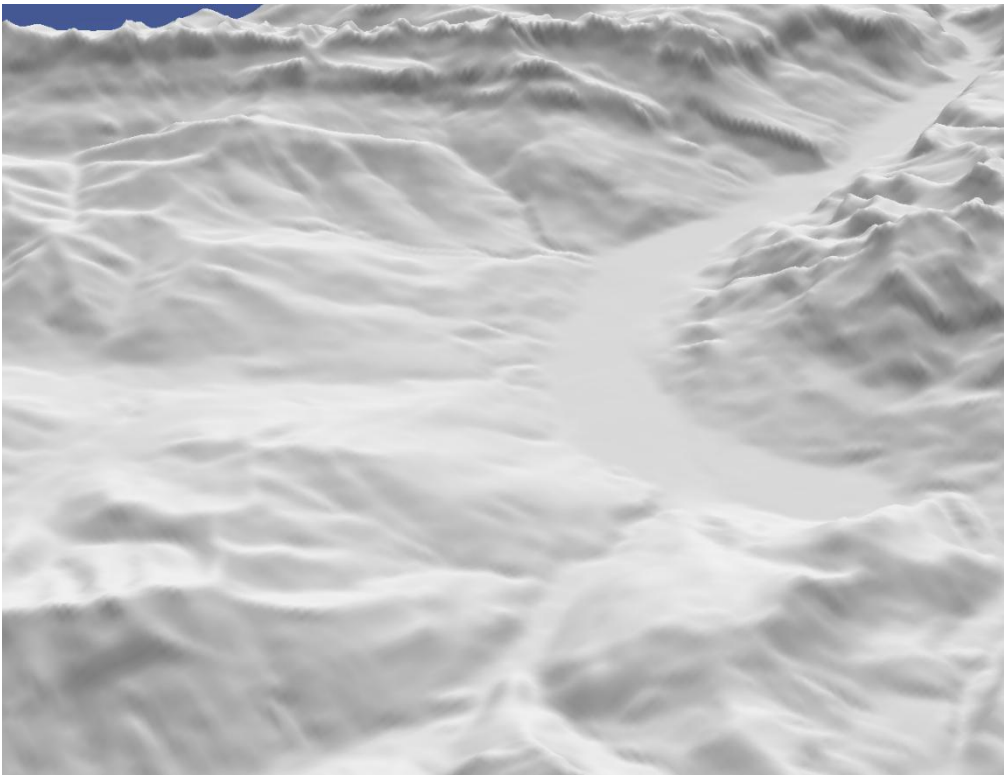
J'utilise une fonction de distribution gaussienne (Bell Curve) pour lisser les bords de la modification (pour pas avoir de truc trop abruptes).



Au-dessus, je faisais monter le terrain avec un radius de 500m et 50m d'intensité (la dernière j'ai un peu spammer).

En dessous, je creuse le sol, sur la dernière capture j'utilise un radius plus grand (1500m).





*Ici j'élève une
montagne (4000m
radius).*

4) Érosion Hydraulique :

Principe de base : Ici je veux simuler le cycle de l'eau pour sculpter le terrain :

1. Érosion : Le creusement de lits de rivières sur les pentes fortes où l'eau accélère.
2. La Sédimentation (Dépôt) : L'accumulation de matière dans les cuvettes et les plaines, tendant à aplanir les vallées.

J'ai fait une simulation itérative sur des lois physique, je traite l'écoulement de manière séquentielle.

À chaque itération (cycle de pluie), je fais :

1. Précipitation (Rain) : Une quantité d'eau est ajoutée uniformément ou aléatoirement sur le terrain.
2. Écoulement (Flow) : L'eau se déplace vers les voisins les plus bas (gradient de pente). La vitesse du flux dépend de la pente.
3. Érosion vs Dépôt :
 - Si le flux transporte plus de sédiments que sa capacité K ne le permet (ralentissement sur le plat) alors il dépose l'excédent.
 - Si le flux transporte moins que sa capacité K (accélération en pente) alors il arrache de la matière au sol.
4. Évaporation : Une partie de l'eau s'évapore à la fin du cycle.

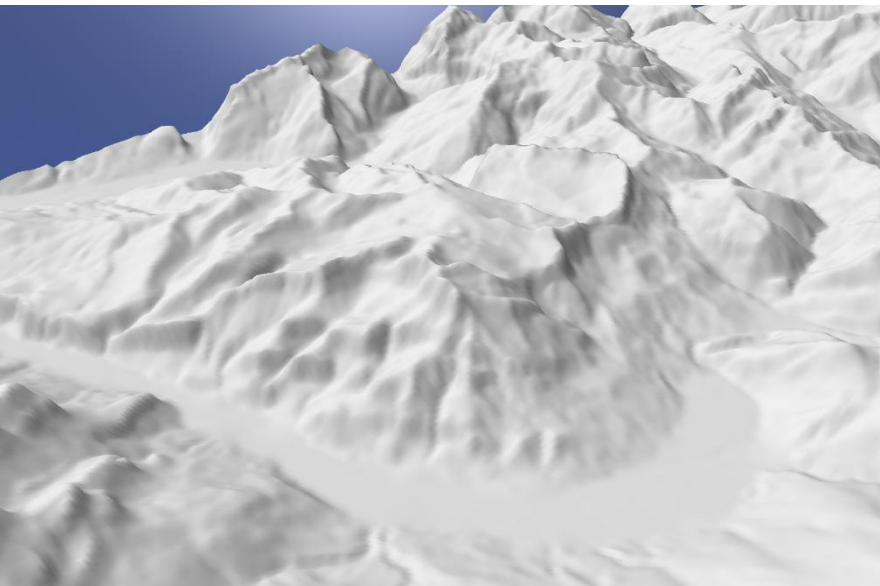
Quand j'ai eu des pics, j'ai fait dans quelques tests juste une passe d'érosion thermique (que je vais présenter juste après cette partie), juste après l'érosion hydraulique. Cela casse les pics et adoucit les pentes.

Performance :

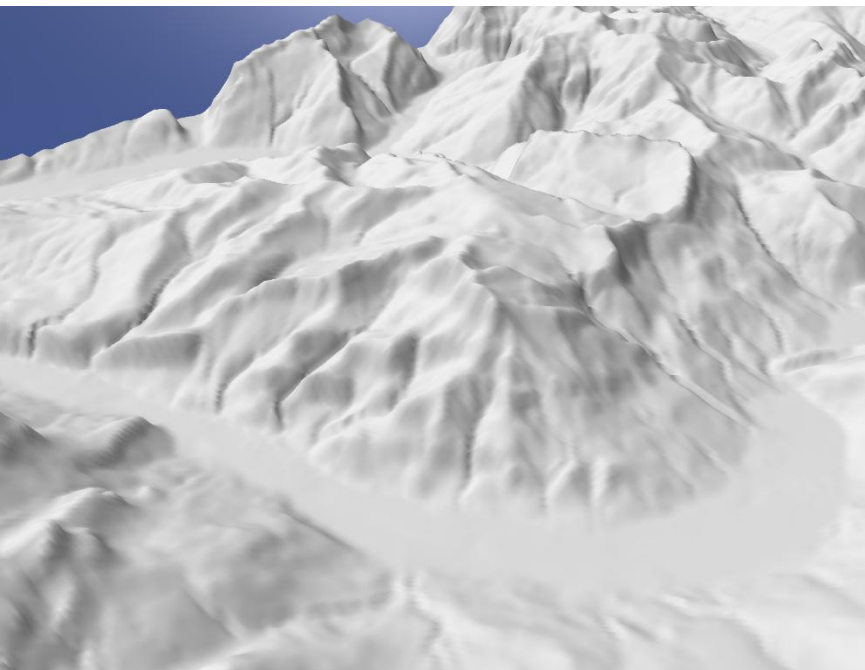
L'algorithme parcourt l'intégralité de la grille ($W * H$) à chaque itération. La complexité temporelle est donc linéaire par rapport au nombre d'itérations N .

$$Temps = O(N * W * H)$$

(Voir les temps d'exécution en ms dans les exemples suivants).



Terrain de base



Temps d'exécution :

1369.99 ms

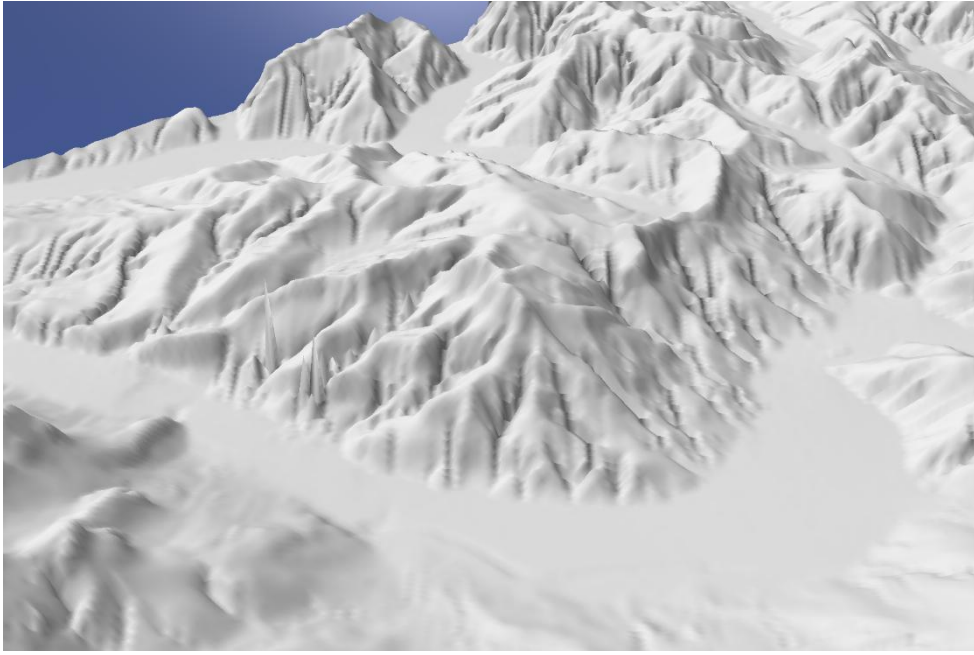
100 itérations

0.01 rain, **0.02** évaporation,

0.5 flowrate, **4.0** capacité,

0.3 erosionRate,

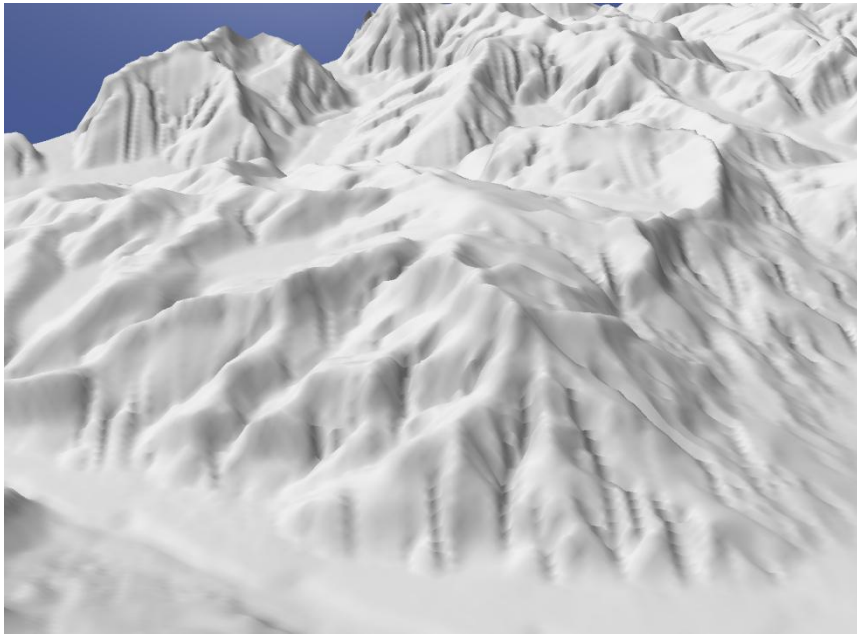
0.3 deposition Rate.



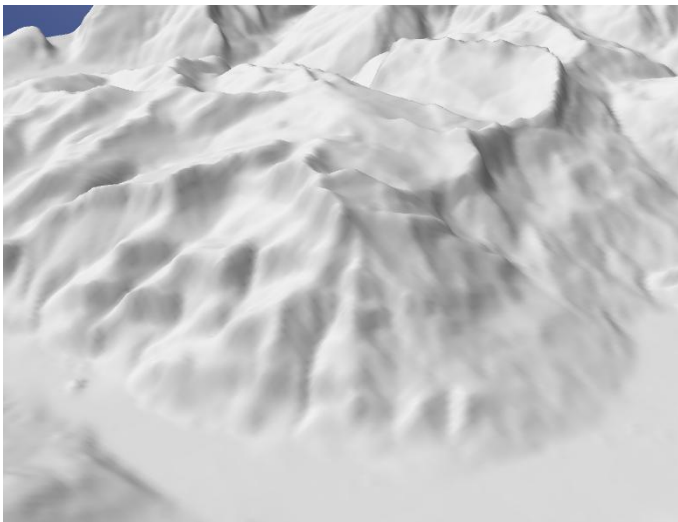
14174.3 ms

1000 itérations

Même variables..



100 et 0.1 en rain



100

0.2 evap

5) Érosion Thermique :

Principe de base : Je veux simuler l'effritement de la roche et le glissement des sédiments secs sous l'effet de la gravité.

Chaque matériau (sable, terre, roche) possède une pente maximale au-delà de laquelle il devient instable et s'effondre.

- Si la différence de hauteur Delta h entre deux cellules voisines est supérieure à un seuil T, de la matière est transférée du haut vers le bas.
- Le seuil T est défini par : $T = \text{TailleCellule} * \tan(\alpha_{\max})$.

À chaque pas de temps, je parcours la grille :

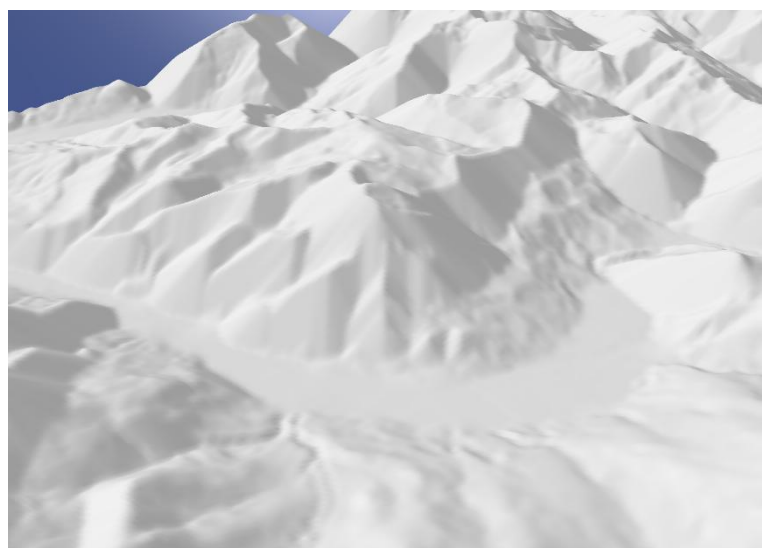
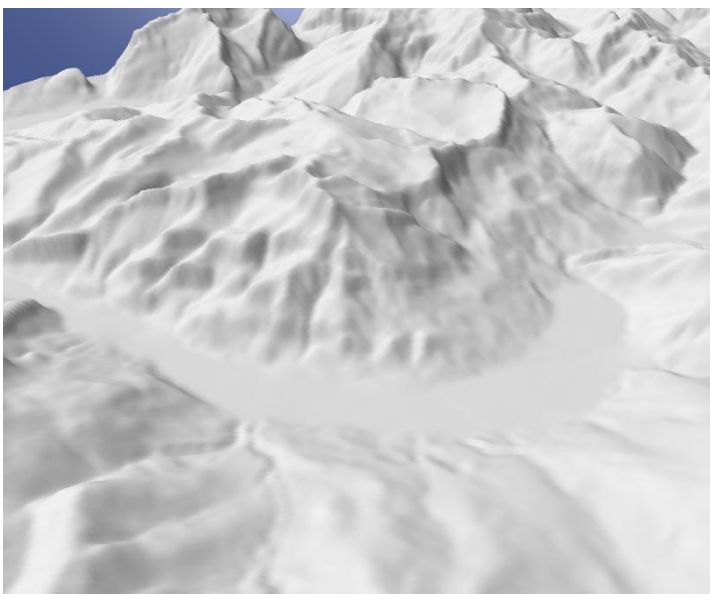
1. Pour chaque cellule (x, y), je compare son altitude avec ses voisins.
2. Si une pente dépasse l'angle de talus autorisé (ex: 40 deg), je calcule l'excédent de matière.
3. Une fraction de cet excédent est déplacée vers le voisin le plus bas.
4. L'opération est répétée jusqu'à ce que le terrain soit stable (ou après un nombre d'itérations, voir ex. après..).

L'érosion thermique peut permettre de nettoyer un peu après une érosion hydraulique :

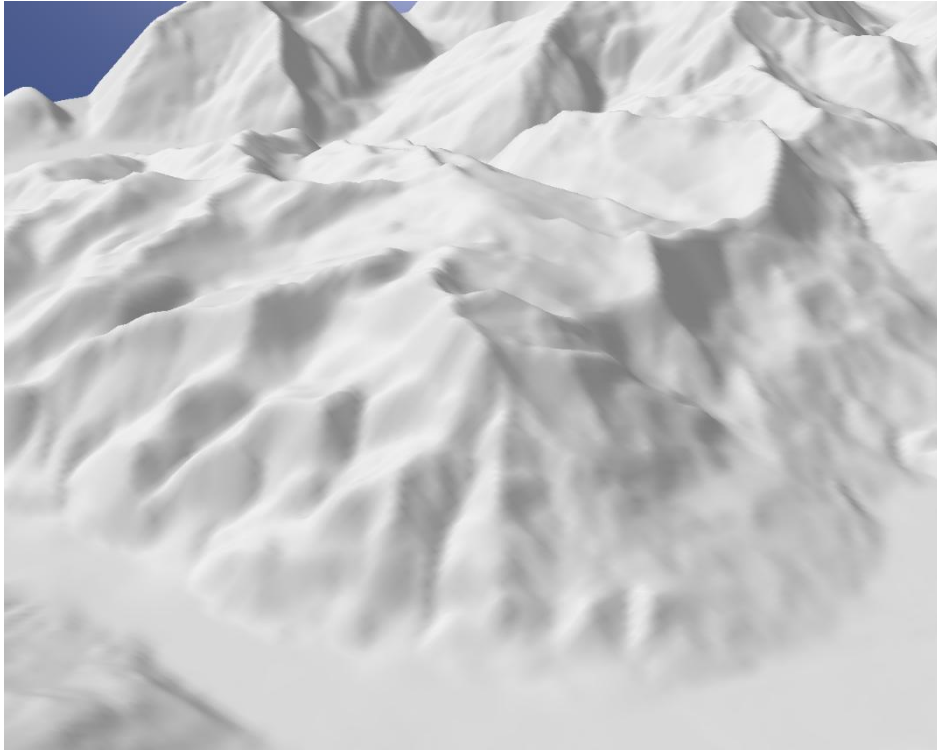
1. L'Érosion Hydraulique creuse des sillons profonds et crée parfois des pics.
2. L'Érosion Thermique passe ensuite pour "casser" ces pics.

Exemple :

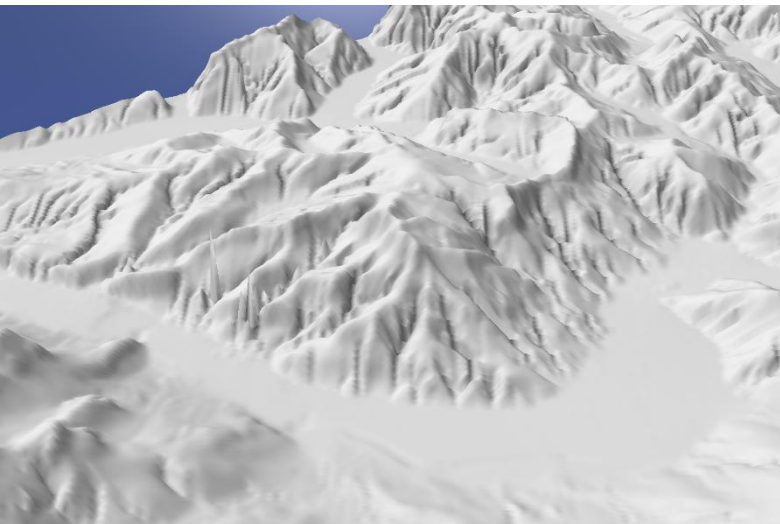
200 itérations , 30° , 0.35 rate



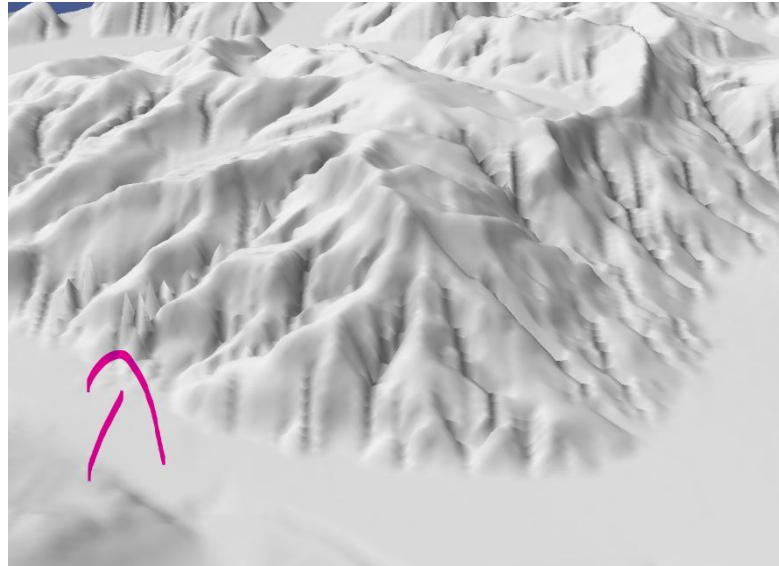
50 itérations , 45° , 0.5 rate



20 itérations , 80° , 0.1 rate



Ici je viens réduire les pics en appliquant ses paramètres



Environ 1ms pour 200 itérations et c'est quasiment instantané pour en dessous de 200 itérations.

6) Végétation :

Le principe de base : Je voulais que naturellement là où les conditions sont favorables (vallées, etc.) cela apparaisse et disparaisse là où l'environnement est hostile (falaises abruptes, etc.).

Je calcule une carte de probabilité de survie $P(x,y)$, comprise entre 0 et 1. Elle vient de la *Loi du Minimum de Liebig*.

La végétation peine à s'enraciner sur les pentes fortes. J'utilise une fonction de transition Smoothstep. Si la pente dépasse un seuil (ex: 35), la probabilité chute à 0.

La végétation préfère une humidité spécifique (ni trop sec, ni noyé). J'utilise une courbe gaussienne centrée sur l'humidité idéale.

Je simule l'ensoleillement en fonction de l'orientation du terrain.

Probabilité :

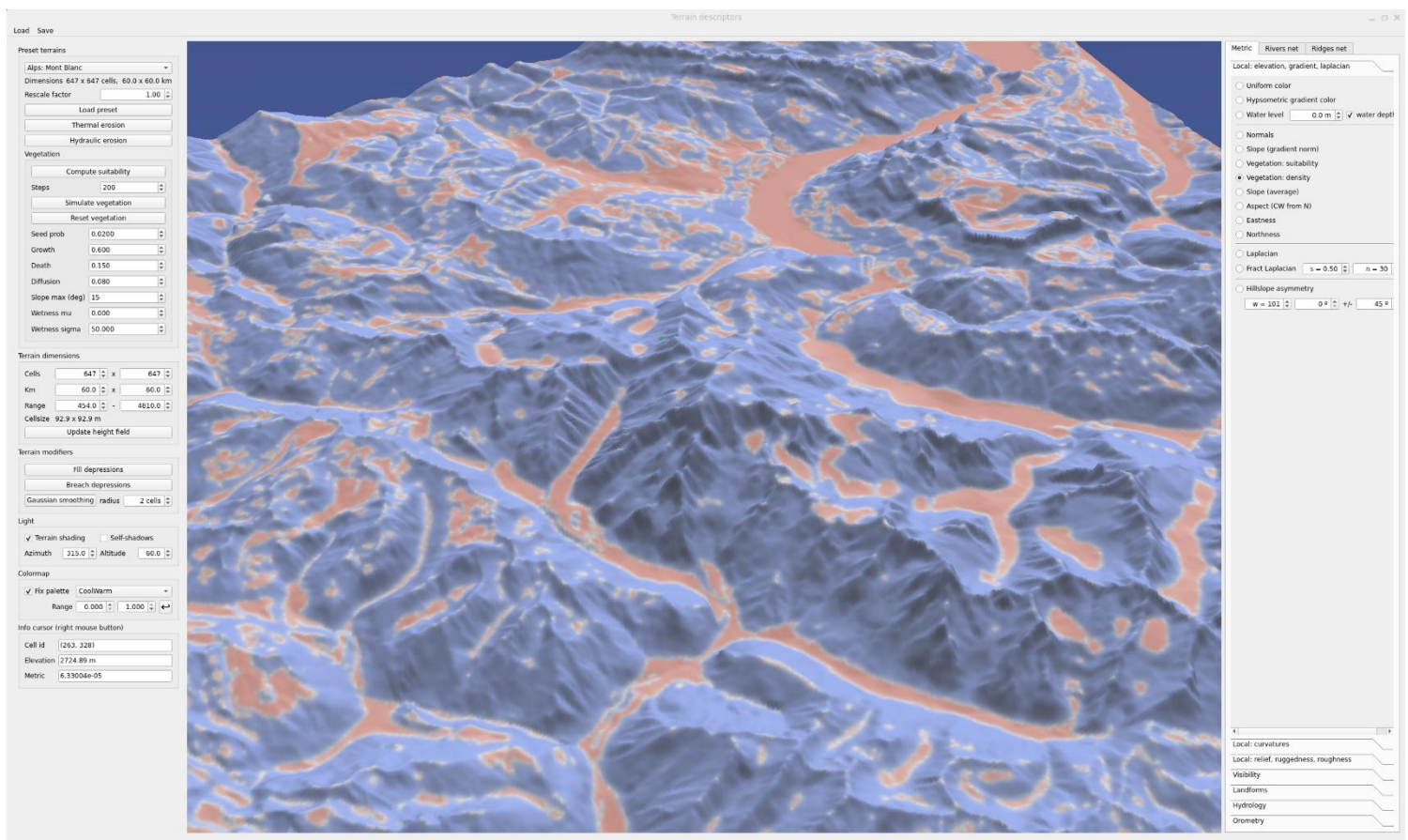
$$P_{\text{global}} = \min(K_{\text{slope}}, K_{\text{water}}, K_{\text{sun}})$$

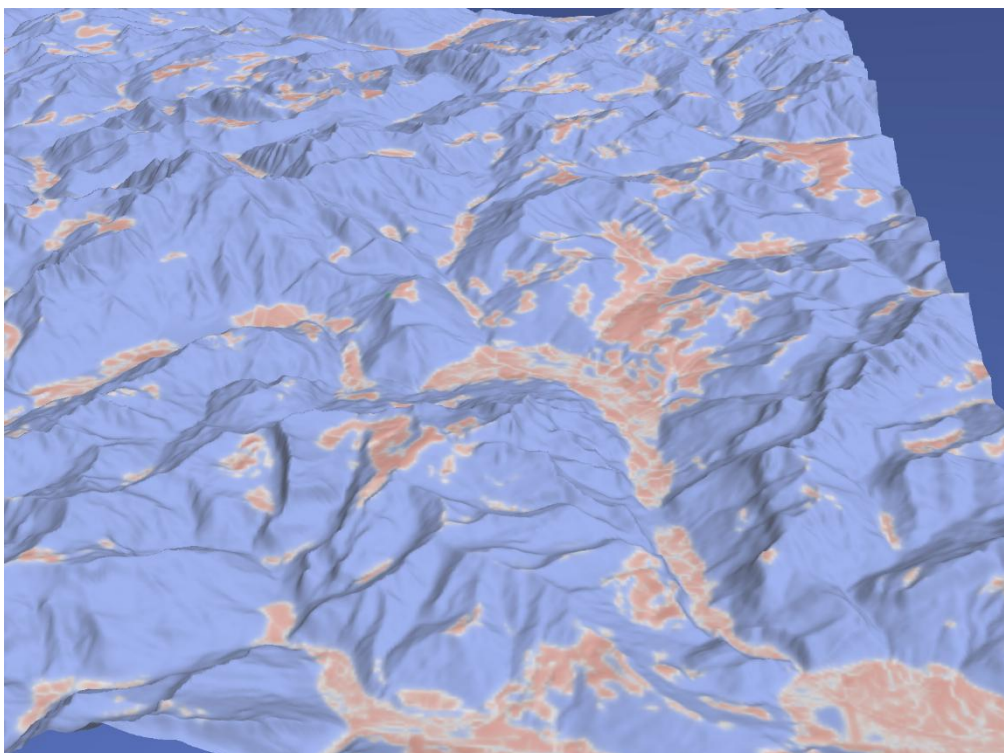
La végétation évolue.

$$V_{t+1} = V_t + \Delta t * (\text{Croissance} - \text{Mortalité} + \text{Diffusion})$$

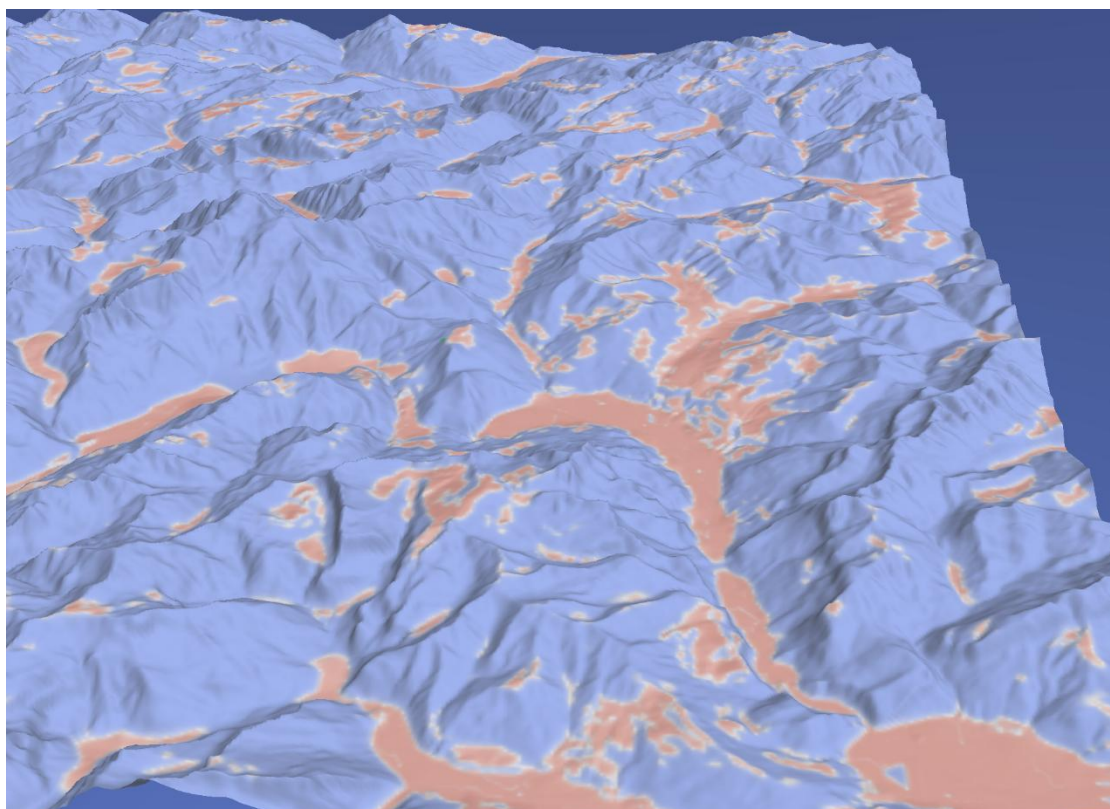
Exemples :

(les paramètres sont visibles à gauche)

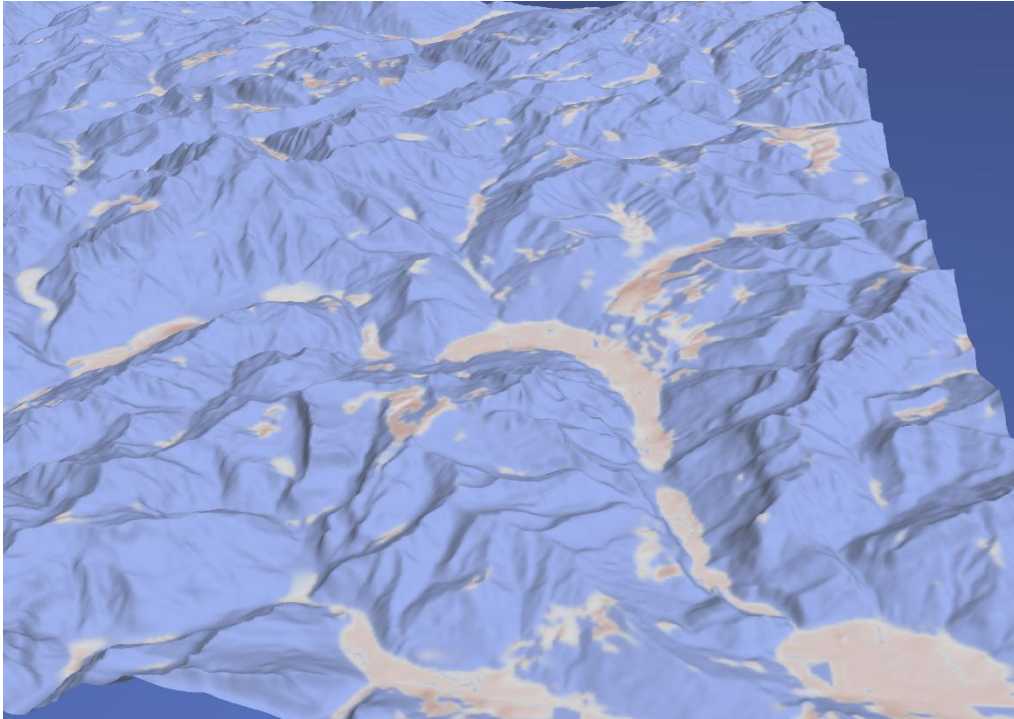




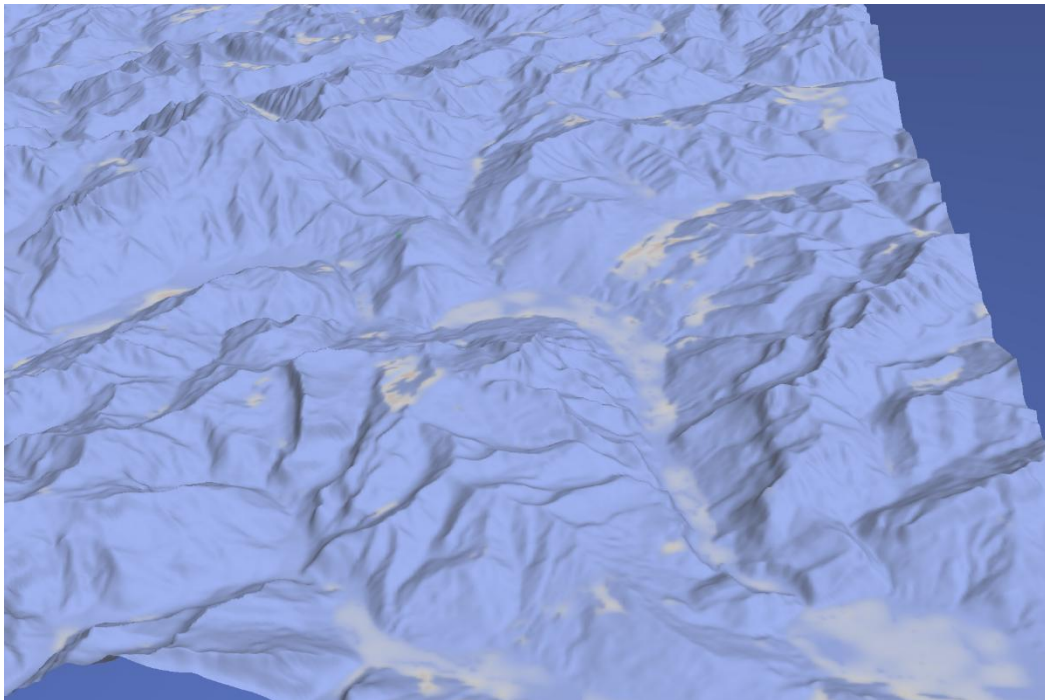
J'ai descendu wetness (sigma) à 5.0 ici au lieu de 50.0 de la première capture.



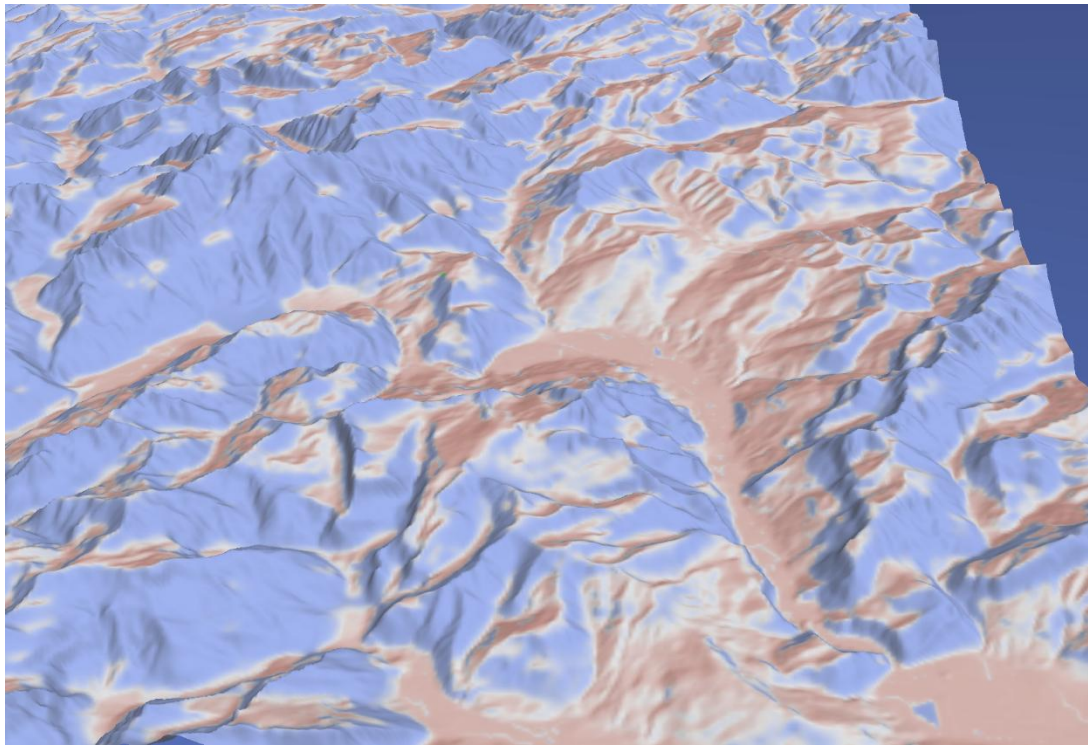
Ici j'augmente Wetness (mu) à 5.0 aussi.



Ici j'augmente la mortalité à 0.55 (on voit tout de suite les répercussions, moins de couleurs donc moins de végétations concentré)

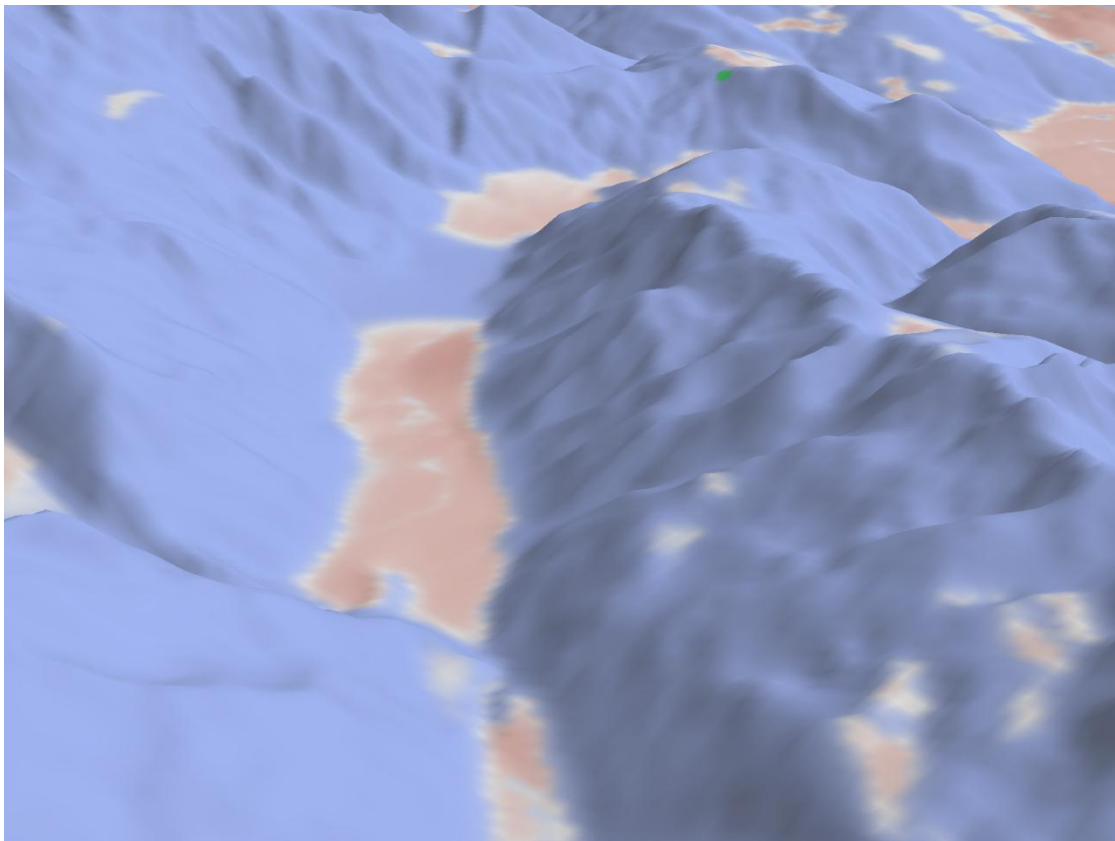


Ici j'augmente à 1.0 la mortalité, si j'augmente encore tout sera mort.

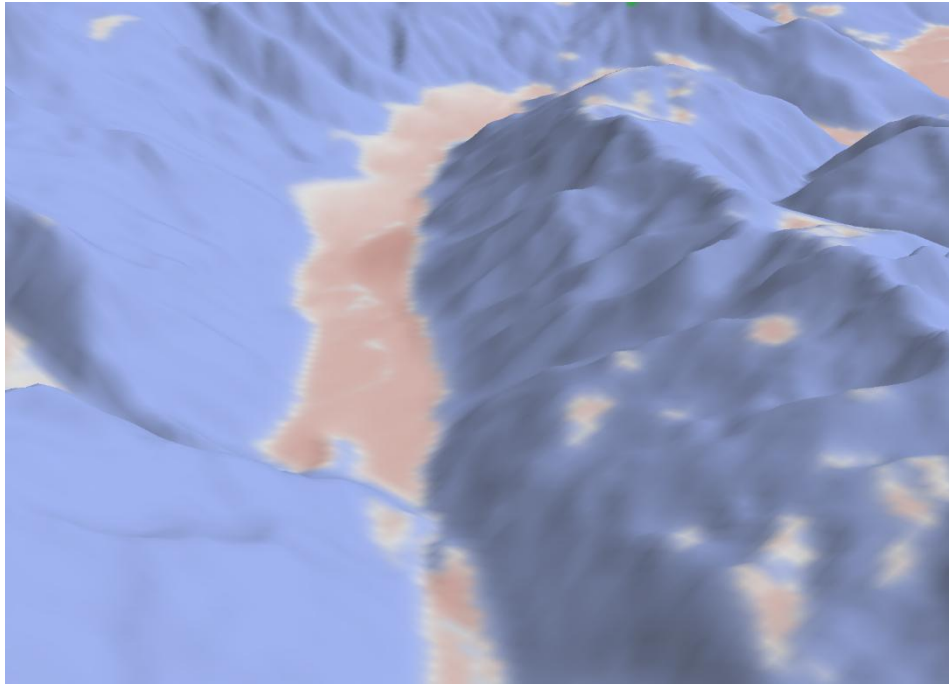


SlopeMax = 30° ici

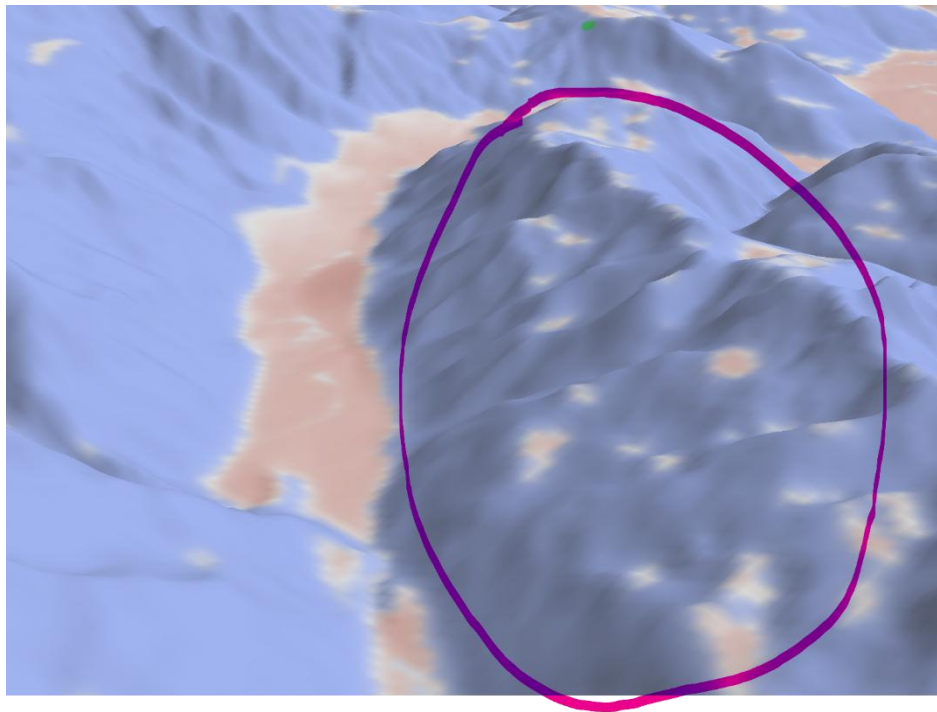
Selon les itérations aussi, itération 1 (60 première itérations..)



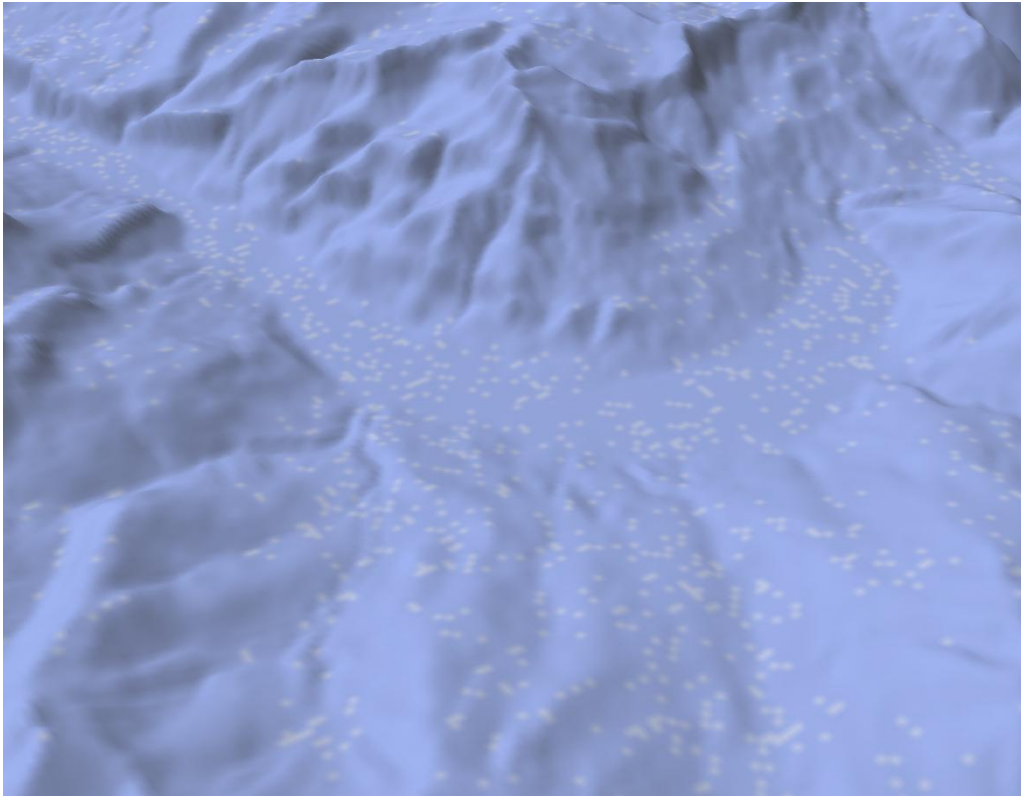
Itération 2



Itération 4-5

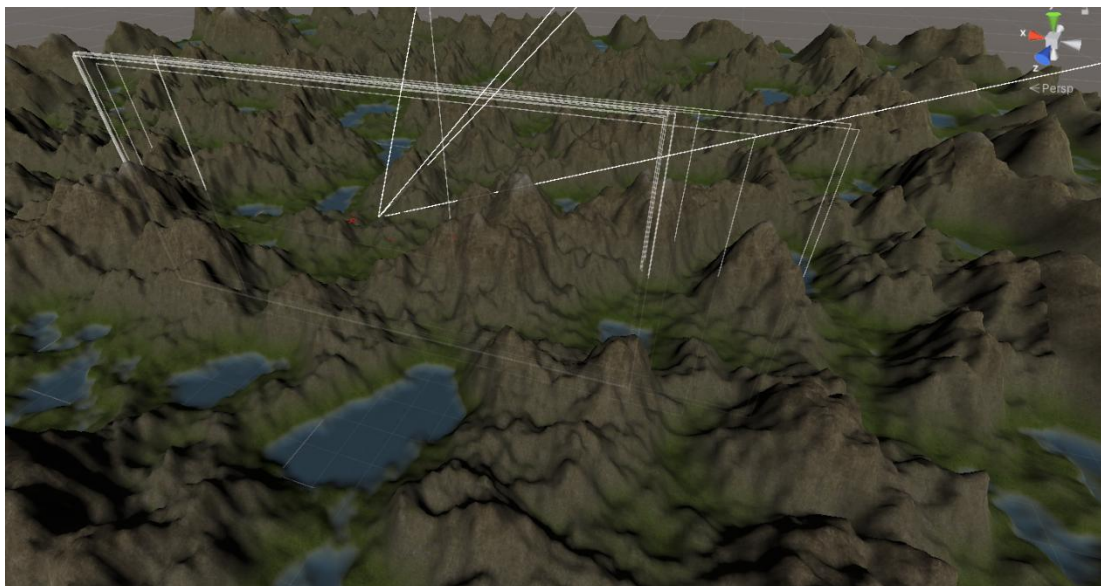


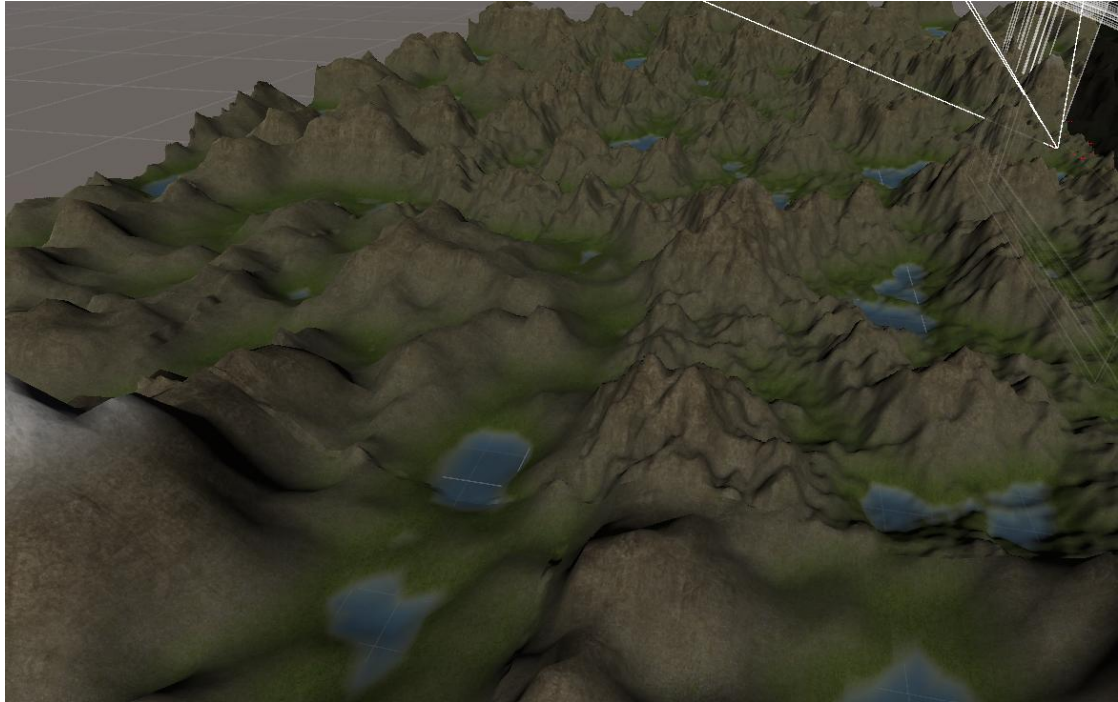
Avant de lancer la simulation, les points que l'on voit sont les positions de SeedProb (plus il est grand, plus il y a de positions possibles pour les graines).



7) En plus :

Vu que ça m'intéressait pas mal j'ai fait une implémentation supplémentaire mais sur Unity, je me suis amusé à créer une map générée procéduralement avec du bruit de perlin et faire pas mal de choses en plus de l'érosion etc, voici quelques résultats :





J'ai implémenté de la LOD aussi. (les chunk de gauche sont loin de la caméra donc pas importante d'être de bonne qualité)

